Adaptive agent modes - the verification loop, not a prompt ladder

What this is. Release 0.3.0 replaces the five-level scaffolding ladder with three modes and one quality mechanism. The modes exist for COST and GOVERNANCE only; correctness comes from the **verification loop**: a harness-authored, digest-frozen suite of independent checks that runs against the agent's work, returns structured failures, and iterates to green under hard budgets. The completion gate re-runs the entire suite itself - a green report the agent wrote is never evidence.

Why it changed. Six admission-gated fixtures and 17 paid calls (including the real medi-ny shadow-replay of a shipped Hermes parser rework) measured a decisive null: prompt-scaffolded arms never beat the unscaffolded control and cost 4-10.7x the tokens. The research corpus had said it from the start - "engineer the loop, not the prompt" (F-2026-07-12-009) - and the audit showed we had implemented that finding as context instead of as a mechanism. The impact-map and adversarial-verification FORMS (agent-filled self-attestation) are removed; their intent is now executed: a deterministic repo-wide symbol sweep replaces the impact map, and independent-verifier findings are ingested into the check suite as blocking checks instead of prose.

Modes (schema 4)

Mode	Use	What you pay for	Profile
lite	Advisory/read-only answers; trivial narrow mechanical or doc changes (<= 4 requirements / 2 files)	Minimal Core, compact ledger, gate	`fast-low-risk`
standard	DEFAULT for all non-trivial mutating work - features, parsers, public contracts, security-sensitive local changes, wide mechanical changes, multi-session work	Precision Context Pack, objective ledger, **verification loop** (frozen check suite + iteration), gate re-run	`balanced-daily`
critical	Critical consequence only: external actions, credential rotation, production data rollouts, cross-system releases	standard + human scope gate + independent verifier (findings ingested as blocking checks) + bounded loop	`architecture-high-risk` + `adversarial-review`

Only the **consequence** axis changes the mode: critical consequence with execution intent selects `critical` and its human gate. Explaining, planning or dry-running a critical action does not. **Clarity** (underspecified task) adds the spec-synthesis capability inside the selected mode - a validated spec with a frozen requirement ledger as an anti-scope-shrink guardrail, whose acceptance tests feed the check suite; it is not a correctness claim. **Continuity** (multi-session) adds durable checkpoints and session handoff. **Breadth** is telemetry only: a 30-file mechanical rename is ordinary standard work - the symbol sweep covers the consumer surface mechanically.

The verification loop

At prepare time the harness snapshots the pre-change workspace and freezes a check suite (`.agentic/check-suite.json`): public tests as acceptance checks, a repo-wide symbol sweep, and any workspace-declared differential/property checks (`harness-checks.json`). The agent implements, then runs

```
python .agentic/verification_loop.py step --suite .agentic/check-suite.json --workspace .
```

Exit 0: all independent checks pass - proceed to the gate. Exit 1: fix the structured failures in `.agentic/loop-feedback.json` and step again. Exit 2: hard stop (iteration budget or no-progress) - escalate to the owner with the remaining failures. Checks and loop budgets are digest-frozen: weakening, removing or reconfiguring them fails the gate; adding checks is always allowed.

Check kinds: **acceptance** (command must exit 0), **differential** (same command on the pre-change baseline and the current workspace; only declared expected changes may differ - silent behavioral regressions fail deterministically), **symbol-sweep** (files referencing touched symbols must be changed or explicitly inspected with a recorded note), **property** (invariants over real data samples - requirements that live in the data are still requirements), **finding** (an independent-verifier finding; resolved only by a harness-re-executed proving command or an explicit owner waiver).

Benchmark protocol

Verdicts require **at least 3 trials per arm** (the medi-ny replay measured ~16-point same-arm run-to-run variance; single trials are exploratory, never verdicts). Loop iterations are counted provider calls. The canary rule is unchanged: an unpaid fixture gets exactly one validated canary call before any main stage, and the canary is not a trial.

Entry points

- `tools/route.ps1 -Adaptive -TaskFile <task>` - route a task (mode decision + Context Pack).
- `tools/adaptive/prepare_context.py` - materialize the context, ledger, baseline snapshot and frozen check suite.
- `tools/adaptive/verification_loop.py` / `tools/adaptive/harness_checks.py` - the loop and its checks.
- `tools/adaptive/pre_submit_gate.py` - the fail-closed completion gate (re-runs the suite).

The classic router (`Router/catalog/modules.json`, default `tools/route.ps1` path) is unchanged; the adaptive layer remains additive and opt-in.